



Introduction to Data Structure

layvathna@gmail.com



Objective

By the end of this course, you will be able to:

- Design, implement and ADTs and algorithms in C++
- Empirically evaluate solutions (operation counts, asymptotic reasoning, quick benchmarks) and defend design decisions with evidence.
- Read, trace, and debug pointer-heavy code safely (memory management, edge cases, off-by-one, leaks).



How the class operate

- Quiz (20%)
- Lesson
- Assignment & Group Discussion + Evaluation & Feedback (40%)
- Final Exam (40%)



Quiz

- At the start of the session (MCQ)
- missing 50% of the quizzes will make quiz score to 0.
- missing 75% of the quizzes will not allowed to do final exam



Lesson

- Review/ Q&A about previous lessons.
- Lesson is Short and Simple
- Focusing on solving the problem
- Participate in the activity will reward bonus scores



Assignment & Group Discussion

- Provide after theory session
- Finalize during lab session and submit before the end of lab session
- missing 40% of the task will make quiz score to 0.
- missing 60% of the task will not allowed to do final exam
- Tips: (use AI wisely)
 - Not about the content (What you submit): 40%
 - Your answer (Q&A and Explanation): 60%



Evaluation

- Goal: verification check
- Assignment:
 - Randomly choose someone to do the task with some modification.
- Group Discussion
 - Randomly choose one of the member to do the task with some modification (team's score)

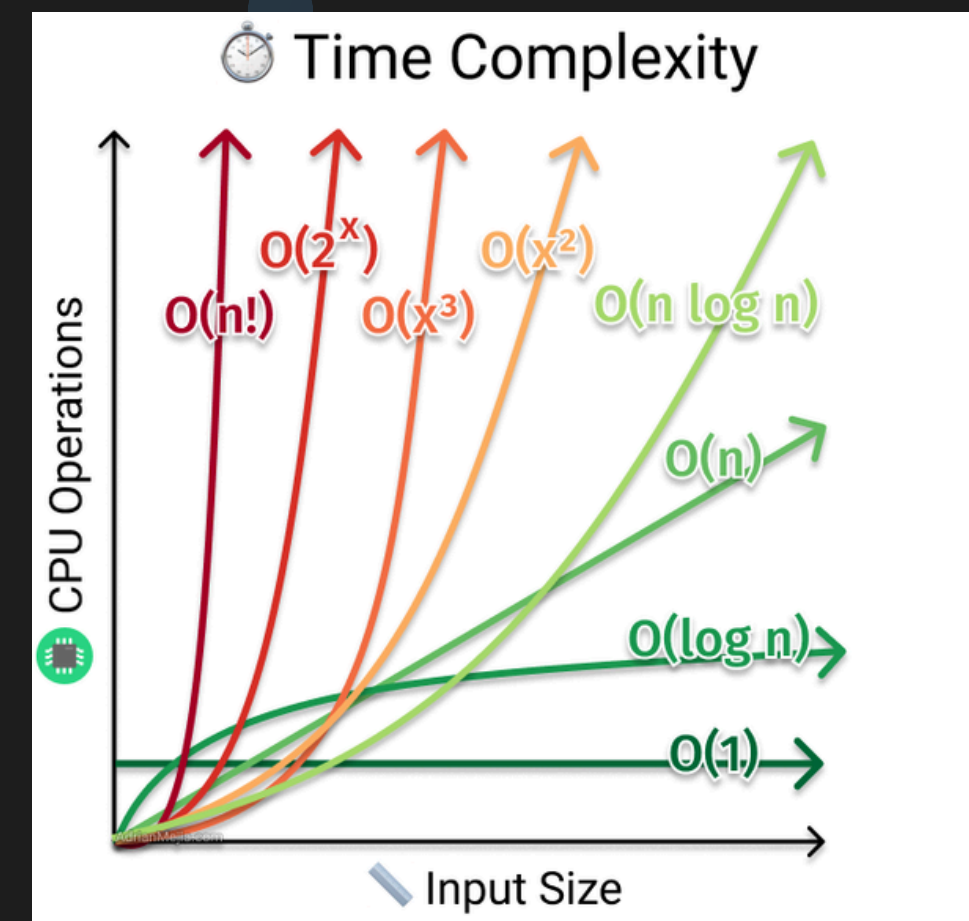


Final Exam

- Paper based
- Not hard coding
- MCQ
- Focusing on finding the best algorithm and implement it

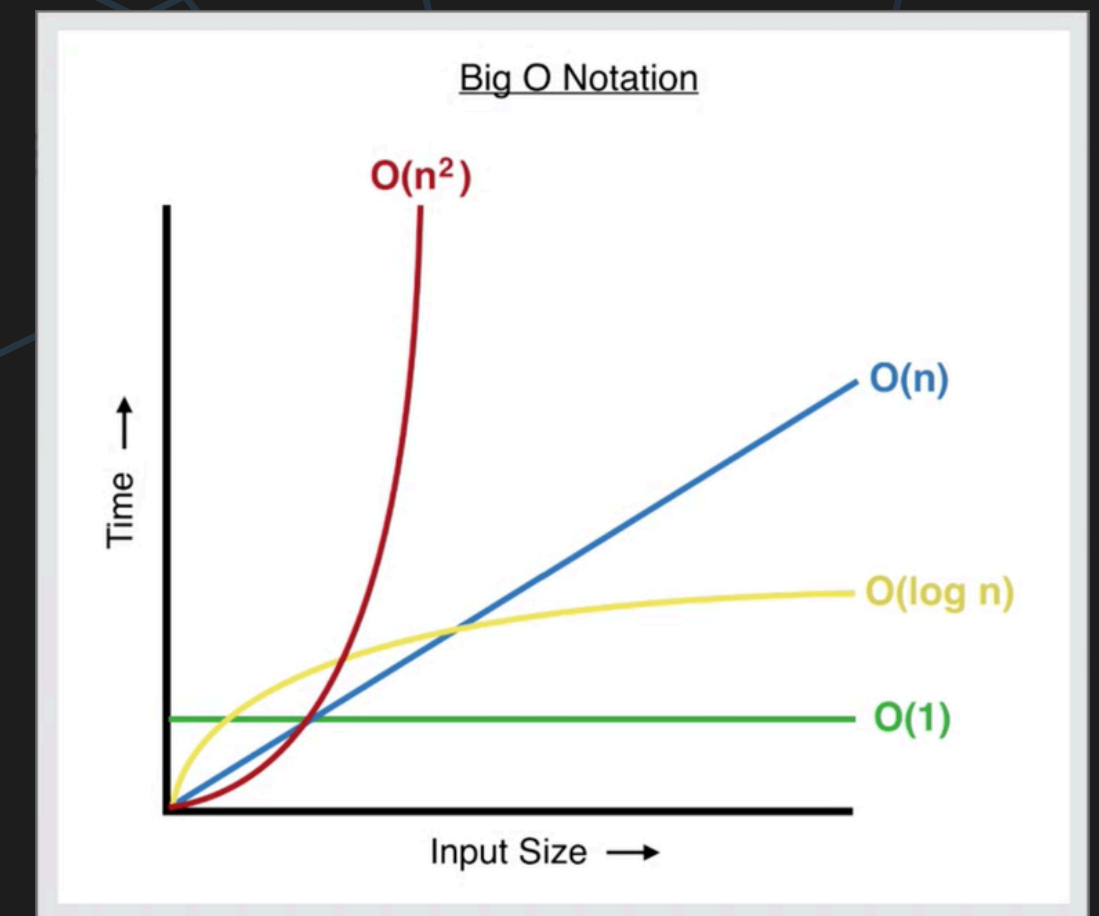
Introduction of Time Complexity

- Definition:
 - Time Complexity measures how the execution time of an algorithm grows with the input size.
- Importance:
 - Helps compare algorithms for efficiency.
- Example Scenarios:
 - Searching for a number in a list.
 - Sorting a set of elements.



Introduction to Space Complexity

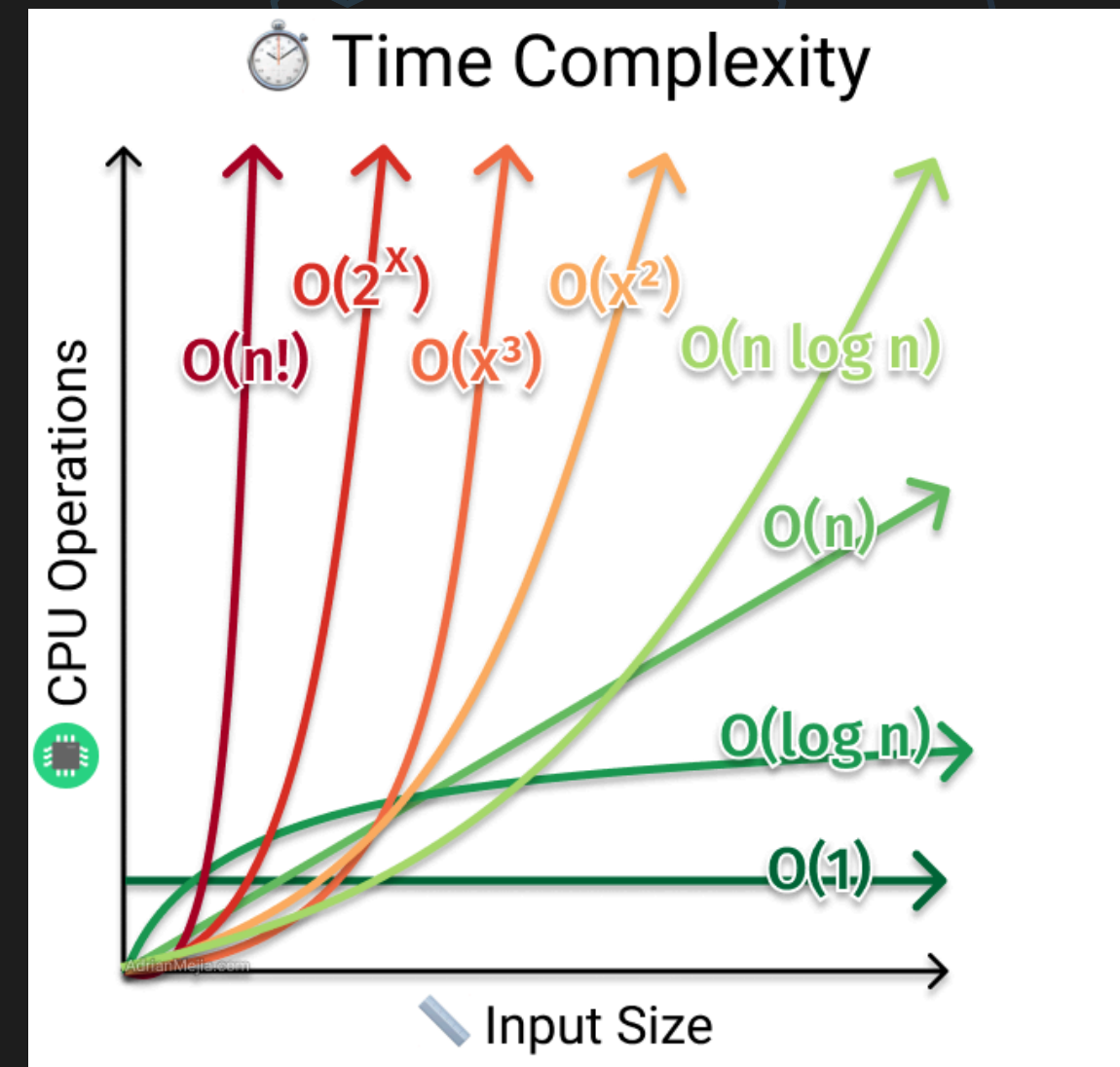
- Definition:
 - Space Complexity measures the amount of memory required by an algorithm, including input data, temporary storage, and auxiliary data structures.
- Importance:
 - Optimizes memory usage for resource-constrained environments.





Basics of Time Complexity

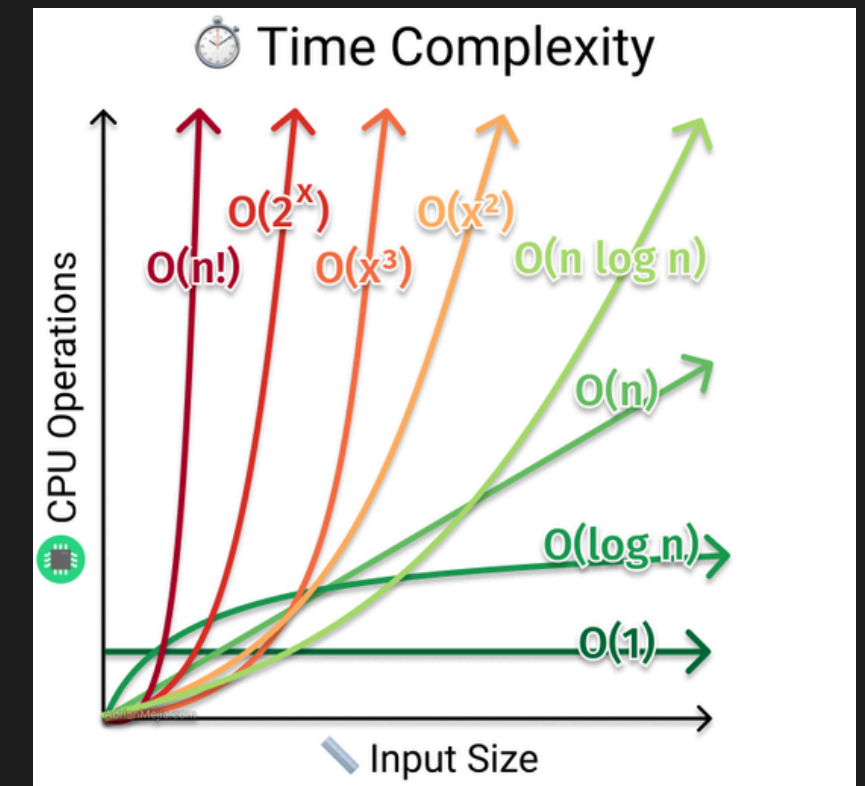
- Common Notations:
 - $O(1)$: Constant Time.
 - $O(n)$: Linear Time.
 - $O(n^2)$: Quadratic Time.





Example (Time complexity)

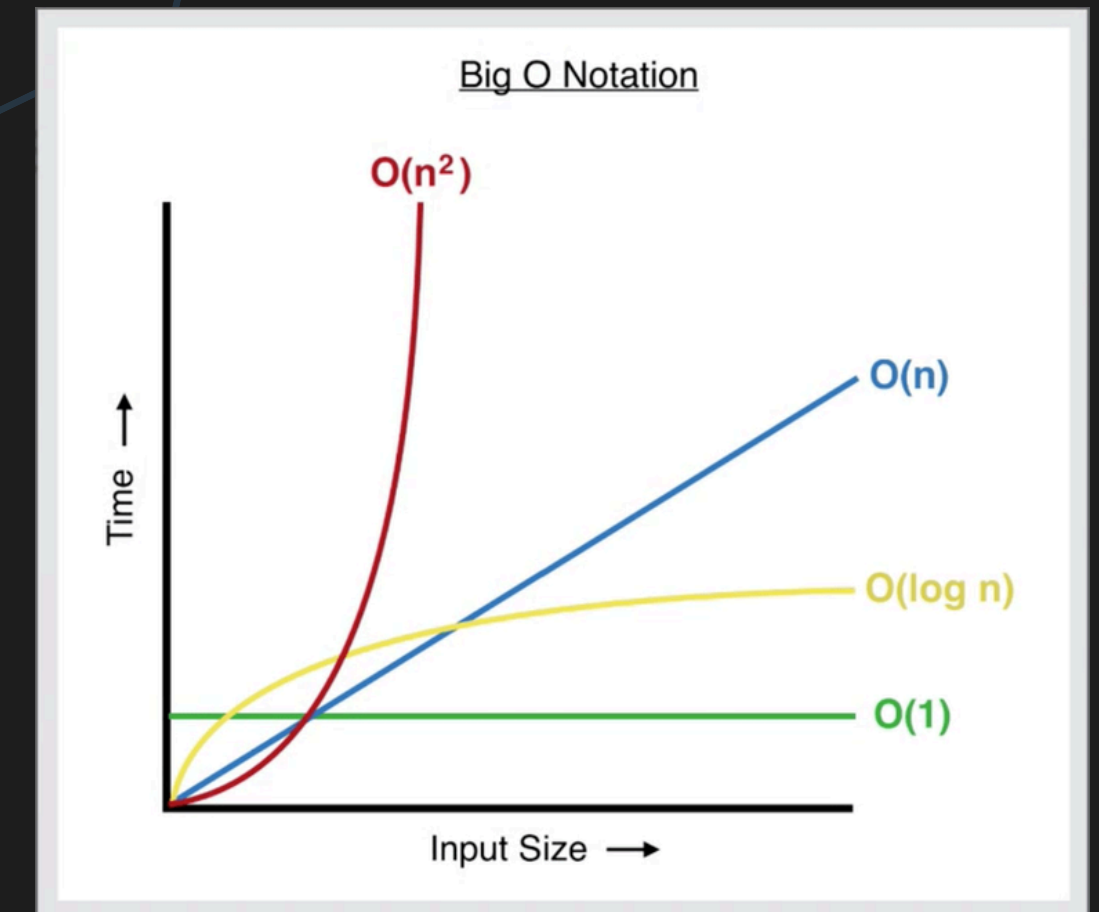
- What is the time complexity of
 - insert 1 element in array?
 - remove 1 element in array?
 - swap 2 element in the same array?





Example (Space complexity)

- What is the time complexity of
 - insert 1 element in array?
 - remove 1 element in array?
 - swap 2 element in the same array?





Summary

- Key Takeaways: After the session you should know
 - How the class is operated
 - Form the group to do the tasks
 - Basics of Time and Space Complexity with practical examples.